Version corrigée du 2026-04-02

À rendre le 24/03/2025

DM Informatique

Consignes :

- Il est conseillé de passer environ **6 h** sur un DM d'informatique.
- Vous préciserez obligatoirement les **spécifications** de toutes les fonctions que vous définirez en python. Ces fonctions seront de préférence commentées. Les spécifications et les commentaires seront d'une autre **couleur**.
- Vous respecterez l'**indentation** des programmes python.

I – Classification d'iris

1 Voir cours : apprentissage supervisé = données étiquetées (algorithme des k plus proches voisins), apprentissage non supervisé = données non étiquetées (algorithme des k-moyennes).

I.1 – Algorithme des k plus proches voisins

2 Mathématiquement,

$$\boxed{d(y_1, y_2) = \sqrt{(\ell_1 - \ell_2)^2 + (L_1 - L_2)^2}.}$$

En python,

```python
def d(y1,y2):
    return ((y1[0]-y2[0])**2 + (y1[1]-y2[1])**2)**0.5
```

3 L'appel `varietes[(6,1.8)]` renvoie la variété `'virginica'`.

4 Le point $(2.5, 0.5)$ est proche des iris de la variété setosa, tandis que le point $(3.5, 1.7)$ est proche des iris de la variété versicolor. Il est donc probable que le premier iris soit classé dans la variété `'setosa'` et le second dans la variété `'versicolor'`.

5 La fonction `voisins` renvoie les k points les plus proches voisins de la donnée x dans l'ensemble E. La liste indices contient les points de E triés par ordre croissant de leur distance à x.

6 L'appel de la fonction `classemaj` avec les arguments donnés renvoie la classe majoritaire `'b'`.

7 Pour $k = 1$, la classe majoritaire est 'setosa' car le point $(2.5, 0.8)$ est proche d'un seul iris de cette variété. Pour $k = 3$, la classe majoritaire est 'versicolor' car le point $(2.5, 0.8)$ est proche de deux iris de cette variété et d'un iris de la variété 'setosa'. Pour $k = 5$, la classe majoritaire est 'setosa' car le point $(2.5, 0.8)$ est proche de trois iris de cette variété et de deux iris de la variété 'versicolor'. Pour $k = 7$, la classe majoritaire est 'setosa' car le point $(2.5, 0.8)$ est proche de quatre iris de cette variété et de trois iris de la variété 'versicolor'. La classe prédite dépend du nombre de voisins ! Les valeurs de k impaires sont utilisées pour éviter les cas d'égalité de voisinage.

I.2 – Qualité des prédictions

8 La précision P est donnée par le rapport du nombre d'iris bien classés sur le nombre total d'iris, soit

$$\boxed{P = \frac{46 + 48 + 45}{150} = \frac{139}{150} \approx 0.93.}$$

9 Voici une version rapide. Voir cours pour version plus longue (et plus simple).

```python
def precision(M):
    """
    Calcule la précision de l'algorithme des k plus proches voisins sur la matrice de confusio
    """
    return sum(M[i][i] for i in range(3))/sum(sum(M[i]) for i in range(3))
```

La fonction `precision` renvoie la précision de l'algorithme des k plus proches voisins. Pour la matrice de confusion donnée, on a `precision(M) = 0.93`.

10 La précision P diminue avec le nombre de voisins k pour les valeurs de k suffisamment grandes, car le nombre de voisins devient trop important et l'algorithme perd en spécificité. La précision augmente avec k pour les valeurs de k suffisamment petites, car l'algorithme gagne en robustesse. La précision est maximale pour une valeur intermédiaire de k. La valeur optimale de k est celle qui maximise la précision, soit k=10.

11 Le point $(3, 3)$ est dans la région de décision de la classe versicolor. Il serait donc classé dans la variété `'versicolor'`. Sans tracer les frontières de décision, il serait moins évident de classer ce point à l'œil nu !

I.3 – Algorithme des k-moyennes

12 La fonction `eloigne` renvoie la donnée x de l'ensemble E dont la somme des dis-

tances aux éléments de la liste est maximale. La fonction `max` renvoie l'élément de E qui maximise la fonction `lambda x: sum(d(x,c) for c in liste)`. La fonction `sum` renvoie la somme des distances de x aux centres de la liste. La méthode `lambda` permet de définir une fonction anonyme.

13
```
def init_centres(E,k,d):
    c = [sample(E,1)]
    for i in range(k-1):
        c.append(eloigne(c,E,d))
    return c
```

Les centres c_2 et c_3 seraient approximativement en $(4.5, 1.5)$ et $(5.5, 2.5)$.

14
```
def minclasse(x, centres, d):
    c    = centres[0]
    dist = d(x,c)
    for y in centres[1:]:
        if d(x,y) < dist:
            c    = y
            dist = d(x,y)
    return c
```

15

16

17
```
def kmoy(k, E, d, n):
    """
    Entrée : k (int) : nombre de classes
             E (list) : ensemble de données
             d (func) : une distance
             n (int) : nombre d'itérations
    Renvoie la classification des données E obtenue par l'algorithme des k-moyennes.
    Sortie : p (dict) : la classification des données E
    """
    centres = init_centres(k, E)
    for j in range(n):
        p = partition(E, centres, d)
        i = 0
        for classe in p.values():
            centres[i] = barycentre([x for x in p if p[x] == classe])
            i += 1
    return p
```

18

19

I.4 – Méthode du coude

20

21

22